



易 灵 思

TinyML开发框架介绍

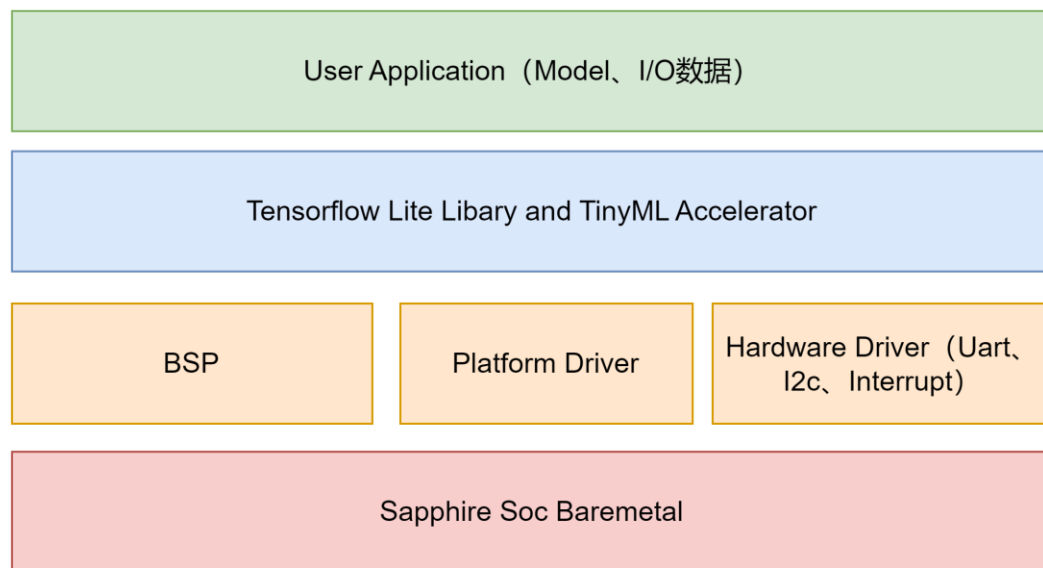
钛金FPGA Ti60F225

国产化中高端FPGA领军厂商 —— 加速您的创芯！

易灵思(深圳)科技有限公司

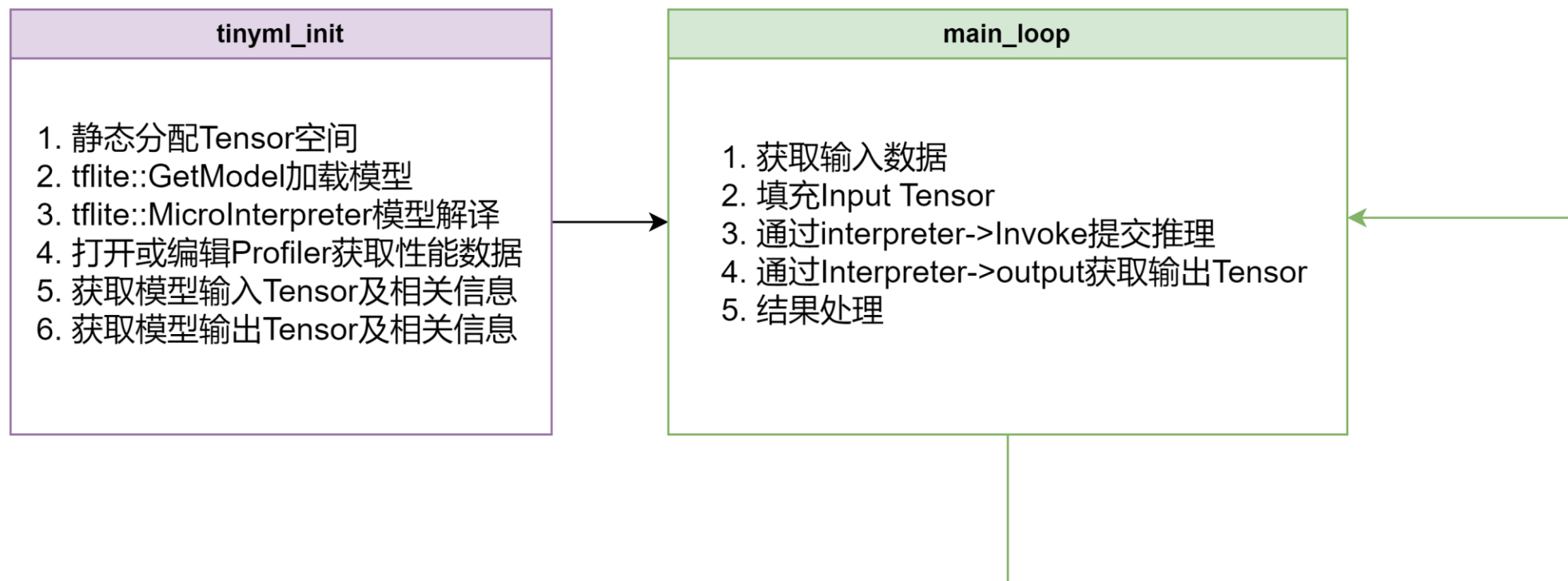
2025.6

Herbert



- 乘法、加法、减法、累加、可变右移操作
- 单通道、双通道、四通道
- 浮点支持: bfloat16 模式运算, 并能实现FMA晕眩。
- 支持符号运算
- 支持级联
- 多个小块 (DSP12/24) 会被合并到同一个DSP48
- 接口寄存器参数可配
- 超高性能FMax可达1GHz

TinyML 应用开发流程图





Tensorflow lite 代码结构

▼	tinymt_ydp
>	Includes
▼	src
▼	model
>	define.cc
>	define.h
>	input_img_000000018380_96x96x3.cc
>	input_img_000000018380_96x96x3.h
>	yolo_person_detect_model_data.cc
>	yolo_person_detect_model_data.h
>	yolo.h
	ref_000000018380_tflite_quant_boxes.txt
>	platform
▼	tensorflow
▼	lite
>	c
>	core
>	kernels
>	micro
>	schema
>	portable_type_to_tflitetype.h
>	third_party
>	main.cc
	makefile

目录	作用
src/model	- 加速器参数动态配置 - 神经网络模型
src/platform	Sapphire Soc平台相关
src/tensorflow/lite/c	Tflite 加速库公共数据类型定义
src/tensorflow/lite/core	Tflite 加速库C++相关工具实现
src/tensorflow/lite/kernel	Tflite 加速库核心算子实现
src/tensorflow/lite/micro	Tflite 加速库 MCU 平台辅助库
src/tensorflow/lite/schema	Flatbuffer 编译器生成的结构
src/third_party	第三方库依赖如： - Flatbuffer - 低精度GEMM加速库 - CPU相关SIMD优化等等

TensorFlow Lite 库结构介绍

```

v tensorflow
v lite
  > c
  > core
  v kernels
    v internal
      > optimized
      v reference
        v integer_ops
          > add.h
          > conv.h
          > depthwise_conv.h
          > fully_connected.h
          > l2normalization.h
          > logistic.h
          > mean.h
          > mul.h
          > pooling.h
          > tanh.h
          > transpose_conv.h
        > add_n.h
        > add.h
        > arg_min_max.h
        > batch_matmul.h
        > batch_to_space_nd.h
        > binary_function.h
        > ceil.h
        > comparisons.h
        > concatenation.h
        > conv.h
        > cumsum.h
        > depth_to_space.h
        > depthwiseconv_float.h
        > depthwiseconv_uint8.h
        > dequantize.h

```

目录(src/tensorflow/lite/kernels/internal)	作用	备注
optimized	SIMD检查	RISCV可忽略
reference/integer_ops	INT8算子加速实现	TinyML主要加速算子
*.h	其余算子通用实现	支持多种格式 INT8/INT16/INT32/FP32等

TensorFlow Lite 加速算子 (Add)

```
C add.h 4
C conv.h
C depthwise_conv.h
C fully_connected.h
C l2normalization.h
C logistic.h
C mean.h
C mul.h
C pooling.h
C tanh.h
C transpose_conv.h
C add_n.h
C add.h
C arg_min_max.h
C batch_matmul.h
C batch_to_space_nd.h
C binary_function.h
C ceil.h

37 }
38
39 inline void ElementWise(
40     int size, const ArithmeticParams& params, const int8_t* input1_data,
41     const int8_t* input2_data, int8_t* output_data,
42     void (*check_arithmetic_params)(const ArithmeticParams&),
43     int8_t (*binary_func)(int8_t, int8_t, const ArithmeticParams&)) {
44     OP_STATUS_T res = add_drv(size, input1_data, input2_data, output_data,
45         params.input1_offset, params.input2_offset, params.output_offset,
46         params.output_multiplier, params.output_shift, params.left_shift,
47         params.input1_multiplier, params.input1_shift, params.input2_multiplier,
48         params.input2_shift, params.quantized_activation_min, params.quantized_activation_max);
49     if(res == OP_OK || res == OP_BYPASS) {
50         return;
51     }
52     CheckArithmeticParams(params);
53     for (int i = 0; i < size; ++i) {
54         output_data[i] = binary_func(input1_data[i], input2_data[i], params);
55     }
56 }
```

➤ 红色为加速代码

- 易灵思统一加速接口

➤ 绿色为原始代码

- binary_func: 本体实现为AddFunc

TensorFlow Lite 加速算子 (Add)

C add.h4

C conv.h

C depthwise_conv.h

C fully_connected.h

C l2normalization.h

C logistic.h

C mean.h

C mul.h

C pooling.h

C tanh.h

C transpose_conv.h

C add_n.h

C add.h

C arg_min_max.h

C batch_matmul.h

C batch_to_space_nd.h

C binary_function.h

C ceil.h

C comparisons.h

C concatenation.h

95

96

97

98

99

100

101

102

103

104

105

106

107

108

109

110

111

112

113

114

115

116

...

inline int8_t AddFunc(int8_t x, int8_t y, const ArithmeticParams& params) {

const int32_t input1_val = params.input1_offset + x;

const int32_t input2_val = params.input2_offset + y;

const int32_t shifted_input1_val = input1_val * (1 << params.left_shift);

const int32_t shifted_input2_val = input2_val * (1 << params.left_shift);

const int32_t scaled_input1_val =

MultiplyByQuantizedMultiplierSmallerThanOneExp(

shifted_input1_val, params.input1_multiplier, params.input1_shift);

const int32_t scaled_input2_val =

MultiplyByQuantizedMultiplierSmallerThanOneExp(

shifted_input2_val, params.input2_multiplier, params.input2_shift);

const int32_t raw_sum = scaled_input1_val + scaled_input2_val;

const int32_t raw_output =

MultiplyByQuantizedMultiplierSmallerThanOneExp(

raw_sum, params.output_multiplier, params.output_shift) +

params.output_offset;

const int32_t clamped_output =

std::min(params.quantized_activation_max,

std::max(params.quantized_activation_min, raw_output));

return static_cast<int8_t>(clamped_output);

}

➤ 原始软件int8 add实现缺点

- 需求多条指令实现（乘法、位移、加减法）
- 每一步都需要量化（量化、反量化）
- 最后需要裁剪输出值
- 串行执行（两个数运算，需求上百条指令）

➤ CI + RTL计算优点：

- 大量计算可使用组合逻辑实现快速计算
- 分步计算可并行化
- 可同时输入多组x、y（无依赖性）

TensorFlow Lite 加速算子 (Add)

```
DEPTHW_INPUT_CNT:8; DEPTHW_OUTPUT_CNT:2.  
CONV_INPUT_CNT:8; CONV_OUTPUT_CNT:2.  
--Hello Efinix TinyML--  
TinyML Setup...  
Total output layers: 2  
Input shape: 4 1 192 192 3, type: 9  
Output shape 0: 4 1 1 1 1404, type: 9  
Output shape 1: 4 1 1 1 1, type: 9  
Done
```

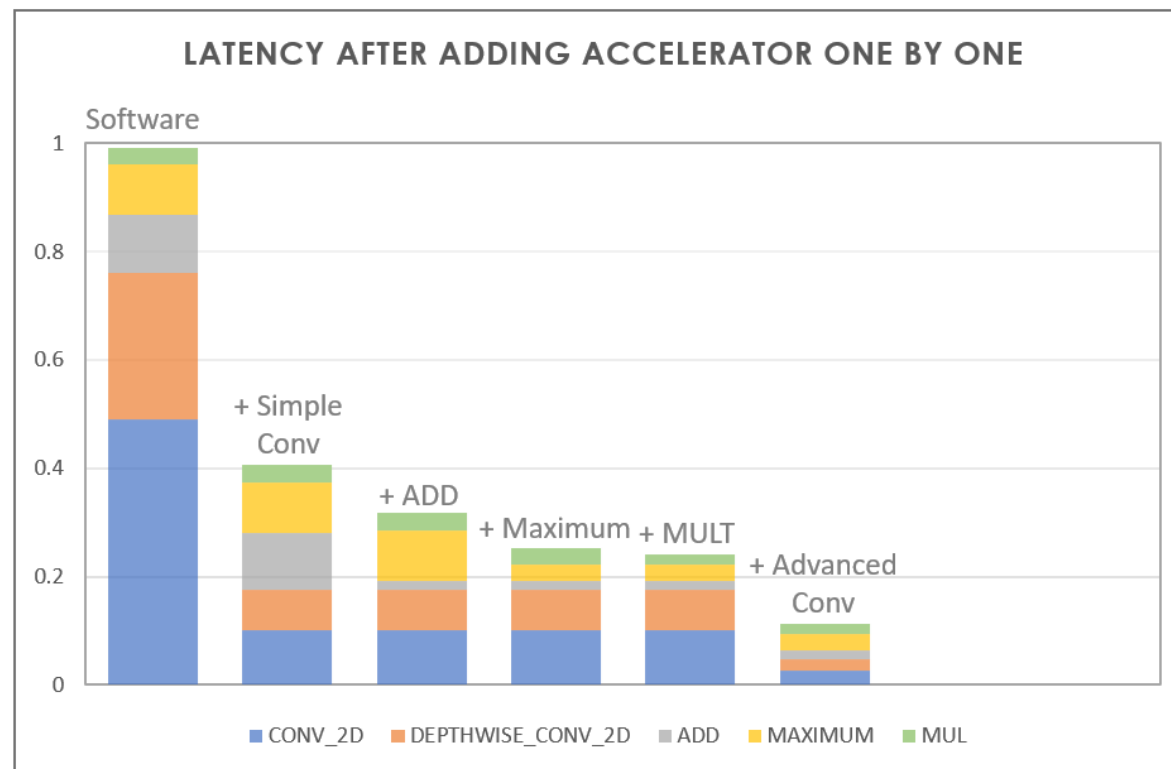
```
Face Landmark Detection Inference...  
0; CONV_2D; STANDARD; 30ms  
1; MUL; STANDARD; 6ms  
2; MAXIMUM; STANDARD; 5ms  
3; DEPTHWISE_CONV_2D; STANDARD; 41ms  
4; CONV_2D; STANDARD; 9ms  
5; ADD; STANDARD; 5ms  
6; MUL; STANDARD; 6ms  
7; MAXIMUM; STANDARD; 5ms  
8; DEPTHWISE_CONV_2D; STANDARD; 41ms  
9; CONV_2D; STANDARD; 9ms  
10; ADD; STANDARD; 5ms  
11; MUL; STANDARD; 6ms  
12; MAXIMUM; STANDARD; 5ms
```

```
DEPTHW_INPUT_CNT:8; DEPTHW_OUTPUT_CNT:2.  
CONV_INPUT_CNT:8; CONV_OUTPUT_CNT:2.  
--Hello Efinix TinyML--  
TinyML Setup...  
Total output layers: 2  
Input shape: 4 1 192 192 3, type: 9  
Output shape 0: 4 1 1 1 1404, type: 9  
Output shape 1: 4 1 1 1 1, type: 9  
Done
```

```
Face Landmark Detection Inference...  
0; CONV_2D; STANDARD; 30ms  
1; MUL; STANDARD; 6ms  
2; MAXIMUM; STANDARD; 5ms  
3; DEPTHWISE_CONV_2D; STANDARD; 41ms  
4; CONV_2D; STANDARD; 9ms  
5; ADD; SOFTWARE; 2592ms  
6; MUL; STANDARD; 6ms  
7; MAXIMUM; STANDARD; 5ms  
8; DEPTHWISE_CONV_2D; STANDARD; 41ms  
9; CONV_2D; STANDARD; 9ms  
10; ADD; SOFTWARE; 2590ms  
11; MUL; STANDARD; 6ms  
12; MAXIMUM; STANDARD; 5ms
```

- 上图为使用profiler后的性能对比，左图为启用加速、右图为不启用加速。该网络中，add层加速超过500倍。

- TinyML提供了卓越的加速性能
- 不同加速器通常能提供数十倍的性能提升
- 可根据模型，灵活配置可用加速资源
- 开发灵活，常规需求通常只需做模型导入和软件开发



感谢您的观看

www.elitestek.com



关注公众号



关注哔哩哔哩

免责声明：

- 1、任何在本文档上出现的信息仅作为参考，实际可能会有变动，请以实际情况为准。
- 2、本文档内任何一页未经允许禁止传播。
- 3、“易灵思”及“Trion”、“钛金系列”等产品均版权归属易灵思（深圳）科技有限公司。